The Judgment Layer

Roberto Pillon Franco

> *When execution becomes free, judgment becomes the bottleneck, so govern the judgment.*

October 2026

The Judgment Layer · Roberto Pillon Franco · October 2026

Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).
You may share and adapt this work for any purpose, provided you give appropriate credit and indicate if changes were made.
creativecommons.org/licenses/by/4.0

Read online and download free: rfranco-br.github.io/Intent-Driven-Flow/book/

Contents

Introduction

PART I · THE GROUND MOVED

1 The Pilot That Never Scaled

PART II · DECIDE WHAT'S WORTH BUILDING

2 Outcomes, Not Output

3 Shipping Is Not Done. Confirmed Is Done.

PART III · SHIP WITHOUT BATCHING

4 The Batch Problem

5 Deploy Is Not Release

PART IV · GOVERN THE JUDGMENT

6 “Who Approved This?”

7 Where Judgment Can’t Be Delegated

8 Your System’s Memory Is Capital

PART V · LEAD THE CHANGE

9 Your Instruments Went Dark

10 How You’ll Know It’s Working

11 From Demo to Scale

12 What It Costs

13 Where to Start, and What We Don’t Know

Introduction

The Driver and the Pilot

A seasoned Driver and a young Pilot stood before a gleaming, chrome vessel at the edge of the atmosphere.

The Driver held a leather-wrapped steering wheel, a heavy brake pedal, and a map of the local highways.

“I’ve spent twenty years mastering the road,” the Driver said. “I know exactly when to hit the gas to overcome friction, when to turn the wheel to stay in the lane, and when to slam the brakes to avoid a crash. I am ready to lead this mission.”

The Pilot looked up at the black expanse of the stars and shook his head.

“Friend,” he said, “where we are going, there are no roads. There is no air. There is no friction.”

The Driver is right about everything he says, and that makes the story useful. He spent twenty years learning when to accelerate and when to brake, and he earned every bit of it. All of it answers conditions that no longer apply. His skill was real, but it was built for friction, and where he’s going there isn’t any.

The assumption underneath everything you’ve adopted

Every delivery framework of the last twenty-five years shares one premise: human bandwidth is the scarce resource.

That premise explains why they all ration. Sprints ration work into what a team can absorb, and WIP limits ration how much can be in flight. Cognitive-load boundaries ration how much of a system a group can hold, and estimation predicts how much human effort a piece of work will consume. The mechanisms differ, and each one served its teams well under the conditions it was built for. Underneath them sits one assumption, that the people doing the work are the expensive, limited part.

That assumption held for as long as the practice has existed. It doesn't hold now. Once execution stops being scarce, rationing it loses its purpose, and each instrument you have for managing delivery rations something.

The argument

When execution becomes free, judgment becomes the bottleneck, so govern the judgment.

"Free" needs a precise reading. It does not mean cheap. AI is not cheap, and if you use it carelessly you will see the expense on an invoice. What collapsed is the human effort and time each piece of work takes, and that changes the shape of an organization. This book is about that change.

The thirteen chapters follow from that sentence, and they come in five parts. Each part names a problem and then says what I'd do about it, so you can read any part on its own. Part I explains why AI pilots succeed and never scale. Part II covers deciding what is worth building and confirming that it worked. Part III covers getting finished work to customers without piling it into batches. Part IV covers governance: who decides, at which moments, and what the organization

remembers. Part V covers leading the change: what to measure, how teams earn autonomy, what it costs, and where to start on Monday.

Where I'm writing from

I have been working at CI&T since 2007. I started as a developer, and today I'm an executive who leads delivery and AI adoption. Most of what I know about running software teams I learned at CI&T, from its clients and its people. Some of the references in this book come from that work, including a sizing method CI&T created that I recommend in chapter 10. I've tried to hold them to the same standard as everything else, and where something comes from my own company, I say so. Weigh it with that in mind.

The ideas in this book come from a framework I have been building and using, Intent Driven Flow, or IDF. Building it was how I learned, and how I tested some of the hypotheses in these pages instead of just arguing for them. I built IDF with IDF: agents did the execution, and people owned the judgment moments this book describes. The book is the argument for leaders. IDF for Teams, the companion work, is the method for the people who run it day to day.

You don't need IDF to use anything here. Adopting a framework like it in an enterprise takes a good amount of change management and a willingness to "fail fast, learn faster". When I say "our own project" in later chapters, IDF is what I mean.

How sure I am

Read this before any chapter.

I'm not sure I'm right about everything, and I'm prepared to be wrong, or only partly right. Some of what follows is measured, some is reasoned, and some is a hypothesis I believe and cannot yet prove. Where the difference matters, I say which is which.

I cite the measured parts, with links, and you should check them. The reasoned parts follow from the thesis, and if you reject the thesis they fall with it. The untested parts are the ones I'd most like someone to disprove. Chapter 13 lists them, including a failure in our own use of IDF that we found only because we went looking.

No controlled study stands behind this book. None stands behind its competitors either, because in 2026 nobody has that evidence yet. Read the argument, check the numbers, and treat confident phrasing as shorthand for a claim, never as proof.

The book's limits

It is a map of where human judgment is non-negotiable and what breaks when it's missing. You won't find a ceremony to adopt or a certification. You decide where to place those moments, what to call them and how formal to make them, and I would expect your answers to differ between a regulated bank and a twelve-person product team.

It stands on its own. You don't need Scrum, SAFe or Team Topologies in place first, and organizations without a named operating model can succeed with it too. I reference other frameworks where they help, and I build on none of them.

It covers more than AI. It says little about models, prompts or tooling, and I expect what it does say to date badly. The subject is what happens to an organization's decisions when the effort

of producing work collapses, and that question will outlive any particular technology.

Who this is for

You, if you can change how an organization decides things: a CIO, a transformation lead, an engineering executive. You have bought the tooling and seen the demos work, and you suspect the operating model underneath hasn't moved.

A delivery team will recognize everything here, but the book gives them no recipe. A companion body of work, IDF for Teams, covers implementation in detail.

Reading it

The book takes about ninety minutes, and it works best in order, though each part also stands on its own. Each part opens with a problem and answers it before moving on.

Each chapter ends with what it costs and with a few questions for your teams. I think the questions matter most. Asking them gets you information you don't have today, whether or not you adopt anything else here.

If you read only one part, read Part II. Its two chapters form the loop the rest of the book builds on: say what should become true for a customer, then check whether it did.

PART I

The ground moved

CHAPTER 1

The Pilot That Never Scaled

Everything worked, and nothing changed

Your organization has run AI pilots, and most of them succeeded.

In two weeks, a small team built something you had budgeted a quarter for. Someone demoed a workflow and the room went quiet. A staff engineer showed you a feature she had built in an afternoon, and you thought, correctly, that this would change how your company builds software. Then you tried to make it the way your whole engineering organization works, and the results stopped resembling the demo.

You have heard the standard explanations, and you have probably given a few of them yourself. The models need another generation, the codebase is a mess, the people need training. Each holds some truth, and none of them explains the gap, because you ran the pilot with the same models and the same people.

Take a pilot apart

Strip a successful pilot down and list what it had. One person, or a group of three or four. The whole context in one head. No dependency on another team's roadmap and no compliance review. And one condition you rarely hear named: the person building it also knew what good looked like.

That last condition carries more weight than the others combined. In a pilot you get judgment for free, because it sits in the room with the work. The builder acts as reviewer,

domain expert and customer proxy at once, and pays no coordination cost, since the whole exchange happens inside one skull. The builder never has to explain the standard to anyone else.

Scale the pilot and you invert those conditions. The people doing the work no longer know what good looks like. Context has to cross a boundary between teams, and someone has to write the standard down instead of carrying it. Judgment cost the builder no effort in the demo. At scale it becomes the most expensive thing you run.

The paradox in the data

Anthropic measured the shape of this in its 2026 Agentic Coding Trends Report.

Engineers in the report say they use AI in about 60% of their work. Asked what they can fully delegate, meaning hand over and walk away from, they put the figure at 0 to 20% of tasks. The report calls this the collaboration paradox and resolves it in one line: effective AI collaboration requires active human participation. Engineers delegate work that is easy to verify or low-stakes. They keep high-level design and anything that needs organizational context.

Those numbers are measured, and I read them as an operating constraint. Sixty percent of your engineers' work now runs through a system in which a human has to stay in the loop for four-fifths of it, and the work does not finish without that person's judgment.

Your pilot had one of those people, with nothing else on their plate. Your organization has a few hundred pieces of work in flight and the same small supply of people who know what good looks like.

Judgment is the part that didn't scale

The models worked in the demo and they work now. You bought the tooling and deployed it, and adoption is probably fine. If you audited every input to your AI program, I suspect most of them would pass.

What failed to scale was judgment: applying it at the moments that matter, through people who weren't in the room when the standard was set.

That explains why AI programs feel hollow from the inside. You did the visible work. You procured tools and trained people, and you measured adoption. You delivered every input. The output you paid for, value reaching customers at a rate that matches the investment, never arrived. None of those inputs touched the constraint.

The argument of this book follows from that: when execution becomes free, judgment becomes the bottleneck, so govern the judgment.

"Free" means free of human effort

AI is not cheap. Used carelessly it costs a lot, and your vendor bills you every month whether or not the work was any good. Several organizations have found that an agent left to iterate without a stopping condition can burn a startling amount of money producing something nobody wanted.

In this book, free means free of human effort and time. A change that used to take three days of a senior engineer's attention now takes twenty minutes of it. You built your operating model to ration that engineer's attention, and the demand on it is what collapsed.

The distinction has practical consequences. If you believe AI made building cheap, you will try to buy your way through the constraint and wonder why the numbers don't move. If you see that AI made building fast and left judgment as slow as it was, you are looking at the right problem.

A leadership problem

If the constraint were technical, your engineers would have solved it by now. They're good, they have the tools, and they have every incentive.

Judgment lives in people, and spreading it across an organization comes down to decision rights and to what people write down. Those belong to you. Engineering excellence can't make up for an organization where nobody can say who decides what, when, and on what basis.

That same fact keeps the problem out of your sight. The dashboards you receive measure inputs: adoption, spend, output volume. None of them tells you whether anyone applied judgment where the work needed it.

The price, up front

The full ledger comes in chapter 12, but I would rather you see the short version now than find it later.

You will have to slow something down on purpose. Each proposal in this book puts back a constraint your tooling investment removed, and people will ask you why, more than once. They will have good reason to ask.

You will have to make tacit things explicit. Standards, decisions and permissions that live in people's heads have to go into writing, because **agents read files and don't absorb culture**.

You will have to let some work be abandoned in public. Once you confirm outcomes, you will see that some of what you built didn't work. You want to see that, and it will still sting in a specific and personal way.

And you will have to give someone authority they may not want, because several judgments in this book need a named person rather than a committee.

I'm not prescribing a reorganization, new roles or a change program. Some organizations will find they need one or more of them to make the change stick. I think that's a fair consequence of taking the argument seriously: you can't improve without changing something. Either way, you will need to decide things you have so far left undecided.

Why this chapter is here

You are living the demo-to-scale gap, and the diagnoses on offer point at the wrong layer. Better models won't close it. Neither will more training or a platform team.

Your pilot ran on a supply of judgment that cost nothing and that nobody could see. To scale, you have to pay for that judgment on purpose, for the first time, at the moment when everything around it stopped taking effort.

Questions for your teams this week

- Name our three most successful AI pilots. For each, what held true that doesn't hold for the wider organization?
- In those pilots, who decided the output was good? Was it the person who built it?
- What have we bought, deployed or trained in the last year that changed how we make decisions, as opposed to how we produce work?
- If a new team wanted to reproduce our best pilot next month, what would we have to tell them that nobody has written down?

Source: Anthropic, 2026 Agentic Coding Trends Report.

PART II

Decide what's worth building

CHAPTER 2

Outcomes, Not Output

The roadmap that describes itself

Open your roadmap, pick any item, and ask what it's for.

In most organizations you'll hear the item restated. "Saved payment methods" is for "letting customers save payment methods." Push once more, and ask what changes for a customer and how you would know, and you'll get a shrug or a story someone invented four seconds ago.

Your people are competent, and I'd blame the format: it produces this answer by design.

The ticket was built for a scarce executor

A ticket allocates work. It tells one person what to do next, with enough precision that they can start without asking. Its whole design follows from one assumption: the person who will execute it is the expensive, scarce part of the system.

That assumption held for fifty years, and I don't think it holds now.

Precise instruction has become the fast part. An agent will break a goal into tasks faster and in more detail than your best analyst, and it will do it again another way if you don't like the first attempt. The hard part, and now the whole job, is knowing whether the goal was worth pursuing. You've automated the very artifact your process is built around, and the process still runs as if you hadn't.

Intents

An intent states three things. First, what needs to be true for a customer that isn't true today. Second, the bet you're making on how to get there. Third, how you will know it happened: where you are today, where you expect to be, and by when.

It names the bet and leaves out the build, on purpose, and that omission makes people uncomfortable.

Ticket	Add a saved-address field to the checkout form.
Intent	Returning customers can complete checkout without re-entering information they've already given us, because we reuse the details we already hold. We'll know it worked when repeat-checkout completion rises from 71% to 80% by the end of the quarter, and support contacts about re-entering details fall from about 300 a month to under 100.

The intent runs longer. It is also the only one of the two that can be wrong in a way anybody would notice. The numbers are what make it possible to be wrong: without a baseline and a target, any movement counts as success. And if nobody can say what the number is today, you don't understand the problem well enough to pursue it yet.

Put together, an intent is a hypothesis: if we make the bet, what we said becomes true for customers, and the numbers show it. Whether it did is the subject of chapter 3.

The intent matters more as execution gets easier

The intuition runs the other way, and it is a reasonable one: if building takes no effort, you can build something else when you get it wrong.

The arithmetic disagrees. When the effort of building the wrong thing falls, you build more wrong things. The work it used to take no longer protects you.

Under the old economics, you killed bad ideas in estimation. Someone said "that's six weeks," and the room reconsidered. That filter is gone. Six weeks became an afternoon, and an afternoon doesn't trigger anyone's scrutiny. You lose your cheapest defense against building things nobody needed at the moment your capacity to build them multiplies.

I don't see anything that replaces that filter on its own. You have to put one back on purpose, and you have to place it where someone states the outcome, because estimation no longer catches anything.

What this buys you

Better solutions than the one you'd have specified. When you say what needs to be true instead of what to build, the agents and engineers doing the work can find approaches you didn't consider. I suspect the saved-address field is often the wrong answer, and the right one is to stop asking for the address twice. A ticket rules that out. An intent leaves it open.

A stopping condition. "Build the thing" ends when the thing exists. "Make this true for customers" ends when it's true, which might take three attempts or might prove impossible. You want to know both, and a ticket-shaped system shows you neither.

The ability to kill work honestly. Cancel a ticket and it looks like a failure. Close an intent as *"not achieved"* and you can treat it as information, because the intent always allowed for the chance it wouldn't work. By changing a format, you change what your culture permits.

What it costs

Writing a good intent is harder than writing a ticket, and you have probably trained nobody to do it. It requires knowing what customers need and committing to a measurable claim about it in writing. Many excellent backlog managers struggle with this, and finding out makes everyone uncomfortable.

It also exposes work with no reason behind it. Some items sit on your roadmap because an executive asked, because they sat on last year's roadmap, or because a competitor has them. When you force an outcome statement onto that work, the missing reason shows up in writing, in front of people. Expect resistance that has nothing to do with the format.

Some work has no customer outcome, and if you pretend otherwise you produce fiction. A compliance mandate, a certificate rotation and a database migration ahead of end-of-life are obligations. Force them into an intent template and you generate the ceremonial nonsense that discredits a framework. Say which work serves an outcome and which meets an obligation, govern the two differently, and don't let anyone dress the second up as the first.

Between the two sits a third kind of work: the enabler. A new payments API, a data platform or an authentication service changes nothing a customer sees, yet some intent can't happen without it. The rule I'd use is that an enabler borrows its direction and its confirmation from the intents it serves.

Before anyone builds it, someone names the intents it unlocks, and it counts as confirmed when those intents are. If nobody can name one, it is either an obligation, and should be called that, or work nobody needs yet. This follows from the argument of the book, not from data, so treat it as reasoned.

Some work changes what customers do mainly for your benefit, such as moving them to a payment method that costs you less in fees. That is legitimate, and it is still an intent, because a customer has to do something differently. But the customer's interest is no longer built into it, so add one more number: the one that must not get worse, such as your renewal rate. Say plainly who benefits, and let whoever approves it decide knowing that.

It also slows the front of the process down, on purpose. While you decide whether something is worth doing, nobody builds. In an organization newly impressed by how fast agents produce things, that will feel like going backward. You'll spend a while arguing that it isn't.

Why this chapter is here

When something goes wrong, someone eventually asks who approved the work, and chapter 6 shows how rarely anyone can answer.

Part of the reason is that you had nothing to approve. You can't approve "add a saved-address field" in any meaningful way. You can only confirm that it sounds reasonable. You can approve a claim about the world: this will become true for customers, and here's how we'll know. Somebody can accept that claim, reject it, or answer for it.

The rest of this book depends on having something at the top of the loop worth governing.

Questions for your teams this week

- Take three items from the current roadmap. For each, what has to change for a customer, and how would we know it happened?
- When did we last stop work because the outcome wasn't materializing, as opposed to because priorities shifted?
- Who writes our intents, and has anyone taught them how?
- How much of the current roadmap is obligation and how much is outcome? Do we govern the two the same way?

CHAPTER 3

Shipping Is Not Done. Confirmed Is Done.

The question that ends the meeting

Your annual review deck says the team delivered forty-seven features. Ask which of them worked, and watch the room.

Someone will name two, the ones with success stories everybody already knows. For the other forty-five, nobody has looked, and you have no mechanism that would make anyone look.

Your delivery system stops measuring too early

Look at where “done” fires in your process. It fires when the work is delivered: merged, deployed, released, marked complete. Your tools work this way, and your reports count work that reached that state.

This started as a reasonable compromise. For most of the industry’s history, confirming an outcome took months while the team moved on within days. Chasing the answer cost more than knowing it was worth. So “delivered” became the proxy for “valuable.” Everyone understood it was a proxy, and over time everyone forgot.

The consequence compounds. Year after year you accumulate an unmeasured backlog of work you shipped and never validated, and you don’t know the ratio because nobody ever asked for it.

You'll have heard the often-quoted figure that some large fraction of software features go unused. The statistic is over twenty years old and methodologically contested, and I wouldn't put weight on it. The bigger problem is that you probably can't produce your own version of it. The number that matters is yours, and you don't have it.

The pile now grows faster

When humans did the execution, teams shipped slowly enough that the unvalidated pile grew slowly. You could ignore it for a decade and mostly get away with it.

Agents changed one side of that equation. Your teams ship faster, and they confirm at the same rate as before. Confirmation depends on customer behavior, which takes as long as it always did, and on someone choosing to look, which nobody has time for.

The gap between what you built and what you know about it widens each quarter, faster than before. You feel it as a leadership team losing touch with whether any of it works.

The change

Keep the intent open after the feature ships.

Move it into a state, call it monitoring or watching or whatever your organization will tolerate, and leave it there until one of two things happens. You **confirm** it: the signal moved, and the thing you said would become true became true. Or you **abandon** it: the signal didn't move, and you decide to stop pursuing it.

Delivery becomes the middle of the story. Shipping the feature is an event on the way to the finish.

Abandoned is a legitimate outcome

This part needs leadership air cover, so I'll be blunt.

I read a portfolio with no abandoned intents as a dishonest portfolio. If everything you attempt succeeds, your success criteria can't fail, or you set your targets where you already were, or somebody decides what "moved" means after seeing the data.

People have to survive abandonment, professionally, socially and in performance reviews. If closing an intent as not achieved costs someone their credibility, you will never see one, and within a quarter the confirmation loop turns into theater. You get the behavior you make safe.

What this buys you

A number you have never had. The share of stated intentions that came true, which tells you what fraction of what you built did anything. Delivery velocity and satisfaction scores can't tell you that. Of the metrics in this book, I think only this one answers the question your board is asking.

Compounding judgment. Confirmation gives an organization its only way to learn which of its beliefs about customers are correct. Without it, twenty years of experience is one year repeated twenty times, with better tooling each cycle.

Permission to stop. You can stop work that isn't moving its signal without anyone having failed, because the intent always carried that possibility. The saving is large, and it never appears in a budget.

What it costs

You will see that a lot of work didn't work. That is the real cost, and the rest is secondary. Your first honest confirmation cycle will probably be unpleasant, and senior people will want to soften it right away. If they win, don't bother starting.

Someone must own the question weeks after everyone has moved on. Confirmation runs on the customer's timeline, and by then the team is three intents downstream. The job belongs to no one by default. Give it to a named person and protect time for it, or it won't happen and nobody will notice.

You also need instrumentation you may not have. "How would we know" is easy to write and hard to answer if the product doesn't emit the data. Some intents will show you that you can't observe your own customers well enough to tell whether you helped them. You want that finding, and you pay for it early.

And attribution is hard, and the temptation to make the results look better than they are is strong. The signal moved, but did you move it, or did the season, the pricing change or the competitor's outage? Real attribution requires holdouts and patience, and most organizations have neither. One rule makes it easier: one bet per intent. If two bets share an outcome and a signal, nobody can tell which one moved it, so give each intent the narrower signal its own bet changes. State your confidence level and resist claiming causation you can't support. I think a confirmation culture that credits itself for every improvement does more harm than having none, because it manufactures false certainty.

Why this chapter is here

Chapter 2 gave you something worth aiming at. This chapter covers the other end of the same arc, where you find out whether the aim was any good.

Everything else sits between the two: the execution, the judgment moments, the release switch. All of it moves work from a stated intention to a confirmed one. If you adopt only two ideas from this book, I'd make them these two. They form the loop, and the rest refines it.

Questions for your teams this week

- Of everything we shipped last quarter, how many outcomes have we confirmed, as opposed to delivered?
- When did we last close something as not achieved? What happened to the person who said so?
- Who checks whether a feature worked, and how long after release?
- For our last three releases, can we observe the thing we said we'd measure?

PART III

Ship without batching

CHAPTER 4

The Batch Problem

More went in, less came out

The current data contains one counterintuitive finding, and I suspect it explains why your AI investment hasn't shown up in anything a customer noticed.

Deployment frequency went down. In organizations where engineering output rose, the rate at which changes reached production fell by 11%, and the time from commit to live rose by 480%.

The figures come from Faros AI telemetry covering 22,000 developers across more than 4,000 teams over about two years, at $p < 0.05$. The deployment-frequency and lead-time numbers draw on about 10% of that dataset, a smaller sample, and you should say so whenever you quote them. Faros sells engineering-intelligence tooling.

On this evidence, your AI program slowed the rate at which change reached customers, and it made your organization feel much busier while it did.

One change, four stages

STAGE	WHAT CHANGED
Writing	PR merge rate per developer +16.2% · average PR size +51.3% · files per PR +59.7%
Reviewing	Median time to first review +156.6% · median time in review +441.5% · PRs merged with no review +31.3% · 25% of PRs now reviewed by AI agents
Shipping	Deployments per week −11% · lead time commit to production +480.4%
Consequences	Bugs per developer +54% · incidents per PR +242.7% · monthly incidents +57.9%

Follow one change through the four rows. An engineer writes it faster, and makes it larger. It waits much longer for review and stands a higher chance of getting none. Then it waits again, longer still, to deploy. When it ships, it ships inside a bigger bundle that fewer people scrutinized, and the incident rate per change has more than tripled.

The loop behind batching

No team sets out to batch. Teams batch when releasing feels dangerous, and the loop feeds itself:

> 1. Releasing feels risky, so your teams release less often and with more ceremony.
> 2. With fewer releases, more changes pile up in each one.

3. Bigger releases carry more risk: more surface area, more interactions, harder diagnosis when something breaks.
4. Higher risk justifies more ceremony and fewer releases.
5. Back to step 2, with a larger batch.

Each step makes local sense, and the people adding ceremony respond correctly to real risk. The spiral is the sum of sensible decisions, so it rarely gets escalated. The people inside it don't feel they are doing anything wrong.

With AI tooling you poured more work into the top of a system already running this loop, and you left the loop alone.

Batch size and blast radius are the same number

A batch of forty changes is a single event, and it carries the combined risk of forty changes released at once. When it fails, you struggle to find the cause, because forty things changed together.

The 242.7% rise in incidents per PR measures what that costs. Bugs per PR rose 28.7%, a real increase but a far smaller one, so code quality explains only part of it. I read the gap between the two figures as the batching effect: changes land in conditions that make failure more likely and diagnosis harder, over and above any drop in the quality of the code.

That changes your risk conversation. Your organization probably treats release frequency as the risk to manage. DORA, the DevOps Research and Assessment program now run by Google Cloud, has published research on software delivery every year since 2014. It has consistently found that throughput and stability tend to improve together, which runs

against the instinct to release less often to stay safe. My reading of the data is that release size carries the risk and that frequency is the lever that controls size. If that reading holds, you have spent years carefully pulling the lever the wrong way.

The cost of seeing it

You probably can't measure your own batch size today. Most organizations track deployment frequency and lead time, two of the DORA metrics, and those are the right place to start. None of the DORA metrics counts how much finished work sits undeployed at a given moment, because a standard toolchain doesn't produce that number. To get it you usually have to instrument something new, and the first reading tends to be worse than anyone expected.

The finding will also sting people who did their jobs well. Conscientious people built the change advisory board and the release calendar to manage real risk with the tools they had. If you present this data as evidence of failure, you lose the cooperation of the people you need most. I would present it as a change in what the evidence supports, with no one on trial.

Knowing this fixes nothing on its own. This chapter gives you a diagnosis. If you stop here, all you've gained is an uncomfortable fact.

Why this chapter is here

Chapter 1 said judgment is the constraint. Here the cost turns concrete: a 480% increase in the time between building something and a customer being able to use it, inside organizations that spent heavily to go faster.

It also sets up what I think is the cheapest intervention in the book. Your teams produce more than ever, so capacity doesn't cause the queue. The release step causes it, because it runs on a fixed schedule no matter how much work arrives. Chapter 5, next, describes what I'd try: separate deploying from releasing, and let the queue drain without anyone working harder. I can't prove that will work in your organization. The mechanism is simple enough that you can test it at low cost and find out.

Questions for your teams this week

- How many completed, merged changes sit undeployed right now? If we can't answer, how soon can we start counting?
- What is our lead time from commit to production, as a median rather than an average, and not the number in the OKR deck?
- Has deployment frequency gone up or down since we adopted AI tooling? Has anyone checked?
- When something breaks in production, how many changes went out in the same release? Could we tell which one caused it?

Sources: Faros AI, The Acceleration Whiplash: AI Engineering Report 2026 · CD Foundation, DORA's 5 metrics (2025) · DORA, research program.

CHAPTER 5

Deploy Is Not Release

The two dashboards that don't agree

Nine months after you approved the AI budget, you are probably looking at two numbers that contradict each other.

Engineering output is up: more code, bigger changes, more merged work. Customer-visible change is flat. Your customers are not seeing change at a rate that would explain the first number.

Most organizations blame review, and you have probably heard that explanation from your own teams. AI writes more, reviewers can't keep up, and the queue sits in code review. That diagnosis is half right, and the half it gets wrong costs the most.

The diagnosis that's half right

Review is under strain. Chapter 4 laid out the numbers: median time to first review up 156.6%, time in review up 441.5%, and 31.3% more changes merged with no review at all. That half of the conventional diagnosis holds.

The fix everyone reaches for follows from the other half, and the other half is wrong. If review were the whole constraint, clearing review would release the work. It doesn't. Deployments per week fell 11% while output rose, and lead time from commit to production went up 480%. Work that has cleared review still isn't reaching customers.

Something downstream of review holds it, and capacity has nothing to do with it. You increased flow into a valve that opens on a fixed schedule, and the queue behind the valve grew. I read this as a plumbing problem, and you pay for it in a 242.7% increase in incidents per PR, which is what you get when larger, less-scrutinized changes reach customers in bigger bundles.

The fusion nobody questions

For the whole history of the industry, teams have welded two events together. Deployment puts code into production. Release lets a customer see it.

Teams fused them because for a long time they had no way to separate them: if the code was on the server, it was live. Release windows, change advisory boards, Thursday-night deploys and rollback plans all follow from that fusion, and they made sense while the fusion was unavoidable. You built the whole apparatus of release management to manage one irreversible moment.

Deployment and release are two events, and you have been able to separate them for years.

Separation in practice

> - Your teams push code to production continuously, as soon as they build and verify it.
> - It arrives switched off, invisible to customers.
> - It sits in the real environment, running on real infrastructure, proven deployable.
> - Later, on its own timeline, a named person decides customers should see it and flips it on.

> - If it goes wrong, they flip it off. Recovery becomes a decision instead of a deployment.

Your engineers will say this is feature flagging and that they already do it. Many do, as a technical convenience for merging unfinished work. The mechanism is the same. The ownership changes: you turn the flag into a governance instrument, and what matters is who holds it and what they answer for.

Three things this buys you

Risk decouples from speed. The trade-off you've been asked to make since your first agile rollout goes away. Teams build and deploy as fast as they can, because customers see none of it. Care comes in at a different moment, and it goes to a different question.

The decision becomes attributable. In the fused model nobody decides to expose a feature, because exposure happens as a side effect of a deployment someone scheduled for other reasons. Ask who authorized a change reaching customers and you get a deployment ticket, which answers nothing. Separate the two and someone decides, at a moment you can name and date, and you can attach the decision to a person. Auditors and regulators will ask you for exactly that.

The batch drains. Finished work stops waiting on the deployment calendar and moves on a human decision that takes seconds, one item at a time. That queue was never waiting for engineering. It was waiting for permission, and you had welded permission to a technical event.

What it costs

These costs are why teams abandon the practice, so I'll name them.

Switches accumulate. Each switch adds complexity to the code and one more thing for someone to keep track of. Left unmanaged, they turn into debt nobody will touch: dead switches nobody dares delete because nobody remembers what they gated. A switch should be live, watched, or removed. Allow a fourth state and you'll spend two years unpicking it.

Someone must own it, and a committee won't do. You need a named, reachable person who is accountable for exposure. I think that is the hardest part, because you are transferring real authority, and the person receiving it usually needs persuading that they want it.

And some systems can't do this yet: monoliths without a configuration layer, mobile apps behind an app store, migrations that can't run ahead of their code. There you face engineering work before governance work. The constraint tends to be smaller than it looks, so measure it instead of assuming it. I expect most delivery surfaces can be made switchable within a quarter, with a stubborn remainder that never will be. You can live with that remainder. Know where it is, govern it differently, and don't let it set the cadence for the rest.

Why this chapter is here

I think this is the cheapest change available with the largest governance return, and I'd start with it in most organizations.

It doesn't depend on a reorganization, new roles or a training program, though some organizations will find they need one of them anyway. You need a technical capability most teams partly have, plus one transfer of authority. With those, you convert the most dangerous moment in software delivery, the

irreversible, scheduled, all-at-once exposure of accumulated work, into a decision that is reversible, continuous and owned.

The judgment always sat in the exposure, never in the deployment. The fused model hid it, and you can't govern what you can't see.

Questions for your teams this week

- What percentage of our delivery surface can we expose independently of deployment?
- Who, by name, decides that a finished feature becomes visible?
- How long does finished work wait between "done" and "customers can see it"?
- How many switches are live right now, and when did we last remove one?

Source: Faros AI, The Acceleration Whiplash: AI Engineering Report 2026. Faros sells engineering-intelligence tooling.

PART IV

Govern the judgment

CHAPTER 6

“Who Approved This?”

The question after an incident

You ship something to customers that you shouldn’t have.

It might be a pricing error, a feature that mishandles personal data, an automated decision that turns out to discriminate, or a change that takes a service down for four hours. The details vary. The question that follows, usually within a day, does not. A board member asks it, or a regulator, a journalist, or your largest customer:

“Who approved this?”

You will want a name and a moment. Most organizations can produce a deployment record, a merge timestamp, and a list of people who were in the vicinity. If you have sat in that room, you know that none of it answers the question.

For a growing share of your work, nobody approved it

Telemetry shows it. The Faros study of 22,000 developers from chapter 4 found 31.3% more pull requests merged with no review at all than before AI adoption. AI agents reviewed another 25% in place of people.

For a growing fraction of what you ship, the honest answer to *who looked at this before customers got it* is nobody, or a machine reviewing another machine’s work. I don’t think either is

wrong in itself. Both become indefensible the moment someone asks you to defend them, because you can't attribute either to a person who accepted responsibility.

Three questions you can't answer yet

Break the governance question into parts and you get three.

THE QUESTION	WHAT YOU CAN PRODUCE TODAY
Who decided this was worth building?	A ticket, written by someone who was told to write it
Who verified it was good?	An automated check result, and possibly a review that took ninety seconds
Who authorized customers to see it?	A deployment record with a timestamp

All three answers are by-products of running a process, and none of them records a decision. They show you that the machinery ran. They don't show you that anybody chose anything.

Friction used to leave evidence

The answers used to exist without anyone building them, because slowness produced them.

Someone had to understand the ticket before starting, so someone had considered the work. Someone spent forty minutes reading the diff, because reading took that long, so someone had examined the work. Someone ran Thursday's release with a checklist, so someone had chosen the exposure.

None of that was governance. It was friction that happened to leave evidence. Remove the friction and the evidence goes with it. There is nothing to decommission, because you never

built anything. You are finding out now that your governance model was a side effect of how slowly people worked.

The regulatory deadline, and the trap in the good news

The dates, as of August 2026:

> - EU AI Act Article 50 transparency obligations have applied since 2 August 2026. You must tell people when they are interacting with an AI system, and you must mark generated content in machine-readable form. A four-month grace period runs to 2 December 2026 for marking on systems already on the market. The EU did not postpone these obligations.
> - The Digital Omnibus, formally adopted and in force since 27 July 2026, deferred high-risk obligations to 2 December 2027 for standalone Annex III systems and 2 August 2028 for AI embedded in regulated products.

This is not legal advice. Jurisdictions differ, and these dates have moved once already.

Most organizations read the second bullet and relaxed, which is a fair first reaction to a deferral. I think it's also the trap.

A deferral removes urgency from work that takes eighteen months to build. You can't write a policy that answers who approved this, on what basis, at what moment. You build that ability into how your organization decides, over many cycles. I suspect teams that treat December 2027 as far off will start around November 2027 and find they needed to start in 2026.

The organizations that pass in 2027 will be the ones that can answer the question in 2026, because they built the answer for their own reasons.

Audit sets the deadline, and learning is the reason

I think compliance framing produces compliance behavior, and compliance behavior produces theater.

If you build a governance model to satisfy a regulator, you will build the cheapest thing that survives inspection: approval fields people fill in, sign-offs nobody withholds, a register nobody reads. It will pass, and it will tell you nothing, because a control that has never stopped anything controls nothing.

The reason to answer the question is more basic. An organization that can't say who decided anything can't learn. When something goes wrong you can't find the decision that caused it, so you can't correct it, and all you can do is add process on top. That's how organizations acquire ceremonies without acquiring judgment. The regulation sets the date by which you'll be forced to notice.

The cost

You can't record decisions you aren't making, and this cost lands earlier than you expect. "Who decided this was worth building?" has an answer only if somebody decided, which requires that someone wrote down something decidable. Chapter 2 covers that, and you need it first.

The work will show you things that nobody decided: projects underway because they sat on last year's roadmap, or because a senior person mentioned them once. When you make decisions visible, you make their absence visible too, in front of the people responsible for it.

You may end up building theater instead. That failure looks like a complete audit trail of approvals that were never in doubt. Chapter 7 has an unflattering example from our own project, the clearest proof I can offer that this failure is easy to fall into and hard to see from inside.

And someone has to accept being named. Attributable decisions put a person's name on an outcome that might go badly. That is a real ask, and I don't think it works without cover from above. Back them when a decision they owned goes wrong, or you'll get decisions owned by committees, which means owned by nobody.

Why this chapter is here

This chapter opens Part IV, and it makes the governance question urgent.

The earlier parts described a system that produces more and ships in larger and riskier bundles. This chapter shows what happens when an outsider asks that system to account for itself.

Chapter 7, next, is my answer: a map of where the question is legitimate and who stands there when someone asks it. It offers no process for manufacturing an approver.

Questions for your teams this week

- Take the last significant production incident. Can we name the person who decided that change should reach customers, and when they decided?
- What share of merged work last quarter got substantive human review? Can our tools tell us?
- If a regulator asked today how we mark AI-generated content in our products, who would answer, and how long would it take?
- Name a decision in the last six months where someone said no to finished work. What happened to them afterwards?

Sources: Faros AI, The Acceleration Whiplash · EU AI Act Article 50 transparency rules · Gibson Dunn, EU AI Act Omnibus Agreement.

CHAPTER 7

Where Judgment Can't Be Delegated

The question that has no owner

Somewhere in your organization right now, an agent is writing code that will reach a customer.

Ask who is accountable for that and you will get an answer. Ask when a person actually applied judgment to it, not just an approval or a signature, and the answer gets vague.

Your people aren't careless. Nobody ever designed the moments where judgment used to happen. They came as side effects of human slowness. A developer thought about whether a ticket made sense because they had to read it before they could start. A reviewer noticed the feature was wrong because reading the diff took forty minutes and gave him time to think. A release manager caught the risky change because the deploy was on Thursday and there was a meeting.

None of those moments were governance. They were friction, and they produced judgment as a by-product. Agents removed the friction, and you lost the by-product with it.

A map, with no procedure attached

This chapter gives you no meeting to schedule, no ceremony to adopt and no RACI matrix.

It gives you a map of the moments where I believe human judgment is non-negotiable: the places where, if you leave judgment out, you get a specific and predictable failure. You

decide where to place those moments in your delivery flow, what to call them, how formal to make them and how many to run. A regulated bank and a twelve-person product team should decide differently.

I am describing what I believe must be true, and your calendar stays yours. The distinction matters. The frameworks that dictated every step got adopted as theater and dropped as overhead. The frameworks people still use twenty years later named the thing that mattered and left the implementation alone.

The four moments

JUDGMENT	THE QUESTION	WHAT BREAKS WITHOUT IT
Direction	Is this worth doing?	The wrong thing, built perfectly, fast
Fitness	Does it solve the problem we stated?	Passing tests, failing customers
Exposure	Should customers see it now?	Risk arrives on someone else's schedule
Confirmation	Did it work?	Output counted as outcome, forever

You met two of these already, exposure in chapter 5 and confirmation in chapter 3. The other two are where most organizations have no cover today.

I should be clear about the status of this list. Four is the number that held up across each scenario I tested it against. That doesn't prove four is complete, and I'm willing to be wrong or partially right about it. Chapter 13 says more.

Direction: is this worth doing?

This one is counterintuitive.

It is a fair instinct to think that when building takes no effort, deciding what to build matters less, because you can build something else. The arithmetic says the opposite. Building the wrong thing got easier, so you build more wrong things. Difficulty no longer protects you.

Under the old economics, you killed a bad idea in estimation. You have probably been in the room when someone said "that's six weeks" and everyone reconsidered. That filter is gone. Six weeks became an afternoon, and an afternoon doesn't trigger anyone's scrutiny.

Without this judgment, failure looks like productivity. Teams are busy, output is high, features ship, and none of it moves anything. I think that makes it the most expensive failure mode in the book: you can't see it as a failure.

Agents will tell you whether something is technically feasible. The judgment you need asks whether the work changes anything that matters to a customer, and how you would know. That question needs someone who owns the outcome. Someone who owns the backlog can't answer it.

The most autonomous example I know of makes the same point. In September 2026, OpenAI ran thousands of agents for 88 hours against the Navier-Stokes problem, one of the Millennium Prize problems, and they exchanged 2.7 million messages with almost no human management. Ethan Mollick described the division of labor plainly: "the agents did the organizing but people decided where to point them, reassessing as the process continued." That is the direction

judgment, applied to the hardest problem anyone has given agents so far. The agents took over the coordination, and the people kept the question of what was worth doing.

Fitness: does it solve the problem we stated?

I suspect this trap catches sophisticated organizations more often than naive ones.

Automated verification has become excellent. Tests pass, security scans clear, performance stays within budget, coverage goes up. The dashboard shows green, and the feature is wrong in a way no instrument could detect, because the instruments check whether the code does what it says. None of them checks whether what it says is worth doing.

Someone has to use the thing. Reading a report about it won't do, and neither will reviewing the diff. Open it, use it as a customer would, and judge whether it solves the problem the intent stated. The work is unglamorous and it scales badly. Organizations that have adopted AI heavily skip this judgment more than any other, because everything upstream got so fast that stopping to use something feels like the bottleneck.

It is the bottleneck, and it's the one you keep.

A warning about delegating this to AI

The obvious efficiency is to have an agent review the agent. It works up to a point, and you reach that point sooner than you expect.

I can be specific, because we ran the experiment on ourselves. We built IDF using IDF, with agents executing, judgment moments in place, and a QA function reviewing each cycle and logging the result. We logged thirteen cycles, and thirteen out of thirteen passed.

I read that result as a sign that the review function had no incentive, no independence and no standing to fail anything. The reviewer and the reviewed sat inside the same system, optimizing for the same completion signal. It caught nothing because it was never going to catch anything.

Automated review multiplies the volume of things you can check. I don't think it replaces the moment when someone whose judgment is independent of the work looks at the work and is willing to say no. A gate that has never failed anything is only recording what passed through it. If your automated review pass rate sits near a hundred percent, treat that as a finding, and don't celebrate it.

Exposure and confirmation, briefly

Chapter 5 made the case for exposure: deployment and release are separate events, and the decision to let customers see something is a judgment with a named owner. It belongs on the same map as the other three. It is a governance decision, and you shouldn't file it as a devops practice.

Confirmation belongs to chapter 3, and organizations miss it more often than the other three. Someone has to look, later, at whether the thing achieved what it was supposed to achieve, and be willing to say it didn't. Without that, you permanently mistake output for outcome, and you run a faster and faster machine for producing changes of unknown value, with no feedback loop at all.

What it costs

Judgment doesn't scale, and you have to stop pretending it will. The rest of your delivery system now scales with compute. This part scales with attention, which is finite and

expensive and belongs to your most senior people. A plan that expects to find efficiency here is a plan to remove the judgment.

Someone has to be willing to be unpopular. A judgment moment that never rejects anything is decoration. You need real people saying no to real work that real colleagues have finished, more than once, and you need to support them when they do. If your culture punishes that, no framework will save you, and I'd bet on this failure mode above all others.

It will feel like friction, because it is friction. You are deliberately putting back a constraint your tooling investment removed, and people will ask you why. The constraint you removed was an accident and the one you're adding is a choice, and you will have to make that argument more than once.

And you will get the placement of these moments wrong at first. Put in too many and you've rebuilt the change advisory board with better branding. Put in too few and you govern nothing. Each team finds its right number by trying, and the number will differ from team to team.

Why this chapter is here

Chapter 6 asked who approved the work and found that nobody could answer. This chapter is my answer: a map of where the question is legitimate. It gives you no process that produces an approver.

Much of this book follows from the map. Outcomes over output is a direction judgment. Confirmed-not-shipped is a confirmation judgment. The release switch is an exposure instrument. The four moments are the framework, and the rest is implementation.

Questions for your teams this week

- At what point does a person decide a piece of work is worth doing, and what happens if they say no?
- When did someone last use a feature before customers did, as opposed to reading a report about it?
- What is our automated review pass rate? If it's near 100%, what has it ever caught?
- Who is allowed to reject finished work, and when did they last do it?
- For the last thing we shipped, who confirmed it achieved anything, and when?

Source: Ethan Mollick, The Dot and the Swarm (One Useful Thing, 1 October 2026).

CHAPTER 8

Your System's Memory Is Capital

An old problem and a new one

You know the old problem. Your best engineer resigns and takes with them the reasons behind a hundred decisions nobody wrote down. Most organizations have made peace with it, and you may have too.

The new problem is stranger. Your agents start each session knowing nothing. To catch up, they read a document with no owner, last corrected by someone who has since left, describing an architecture you replaced in March. They won't tell you it's stale. They will read it, believe it, and produce work that is consistent with itself and wrong.

Culture used to transmit itself

Organizational knowledge has always come in two forms. People write a small amount down. They carry most of it in their heads and pass it on by being nearby. New engineers absorbed it by sitting near experienced ones, having their work corrected, overhearing arguments and noticing what got approved. Nobody planned it. It came as a by-product of people working together, and it took no effort.

Agents don't absorb culture. They read files.

If you haven't written something down, it doesn't exist for most of the agents and people executing your work. Few people talk about this consequence of agentic delivery. It lands

on you as an unfunded mandate: for the first time, your tacit knowledge has to become explicit before it can work at all. That is real work, it's on nobody's roadmap, and I think it is the hidden cost inside every AI adoption program that gets reported as a tooling budget.

Three assets, one balance sheet

I'd treat it as capital, because it behaves like capital. It grows when you maintain it, it depreciates when you don't, and it decides what your organization can do next.

ASSET	WHAT IT HOLDS	WHAT IT LOOKS LIKE WHEN IT FAILS
System memory	What is true about this system: architecture, decisions, conventions, and the reasons behind them	Confident work built on an architecture you no longer run
Practice	How things are done here: patterns, standards, what "good" looks like in your context	Output that passes every check and fails review
Authority	What each agent may touch: tools, data, systems, and who owns that grant	Nobody can state the blast radius of anything

I suspect most organizations have some version of the first, an accidental version of the second, and none of the third.

It depreciates in silence

Technical debt announces itself. You see it in bugs, incidents and slow builds, and eventually nobody can deny it.

Context debt makes no noise. A stale document throws no error, and an out-of-date convention fails no test. An agent reading a wrong fact doesn't hesitate, flag uncertainty or ask a colleague. It proceeds with confidence and produces work that is coherent, well-formed, and built on something that stopped being true two quarters ago.

The output looks correct, because the agent behaved correctly against a wrong picture of the world. You will struggle to catch that, because everything about it looks right.

A second, faster version of the problem happens inside a single working session. Long agent sessions compact their own context as they run, summarizing and dropping detail to make room, and the details that mattered go first: the constraint you agreed on three hours ago, the approach you already tried and rejected, the reason you ruled out a shortcut. The rule I'd draw from this is unglamorous and absolute. If a decision has to survive, write it into the artifact. The conversation won't keep it.

The argument for your next vendor conversation

Your choice of model is temporary, and your captured context lasts.

The models will change. They changed twice while you read about them. The tooling will change, and the vendor you standardize on this year may not be the obvious choice in eighteen months. The prompts, configurations and platform-specific scaffolding you build around them have a short, unsentimental half-life.

What survives each of those transitions is the written-down knowledge of how your systems work, what your organization means by good, and who may touch what. You can carry it across models, vendors and generations of tooling.

The next time the budget comes up, someone will frame it as a tooling conversation, which is a fair starting point. I'd reframe it: tooling spend is an operating expense, and context capture is an investment. You will repeat the first each year. The second compounds.

What it costs

Writing it down is work nobody wants, and teams push it down the list. It competes with delivery, nobody sees it when it's done well, and it has no natural champion. Leave it to each team's priorities and it will lose.

It has no natural moment for maintenance, so you have to create one. Documentation decays because nothing triggers a correction. You need a trigger: an event, a checkpoint, or a named obligation attached to the work. If maintenance depends on someone remembering, nobody will maintain it.

Over-documentation is a failure mode, and it only looks safe. A two-hundred-page context document is as useless as having none and costs more, because now it's stale in ways nobody can find. Aim for the smallest set of things that must be true, and prune hard. Adding feels productive and takes no discipline. Removing is the discipline that keeps the document alive.

If everyone owns it, nobody does. You need a named owner with the authority to delete what other people wrote. The job is unpopular, and nobody will volunteer for it.

And the authority register won't come for free. Permissions accumulate: an agent gains access it needed once and never gives it back, and nobody notices because nothing breaks. Someone has to reconcile what agents can reach against what you granted them, on a schedule. Expect the gap to be uncomfortable the first time.

Why this chapter is here

Chapters 2 to 7 described the loop: state an outcome, execute it, judge it, confirm it.

This chapter covers what makes the loop repeatable. You can run the loop once through effort and attention. To run it fifty times across teams, with people joining and leaving and without quality slipping, your system has to remember what happened between cycles.

It also bridges to Part V. My reading of why most AI pilots don't scale: the pilot ran on the founding team's shared memory, which nobody wrote down, so nobody could hand it to anyone else. The loop itself is not the hard part.

Questions for your teams this week

- If we changed model vendors next month, what would we lose and what would survive?
- Who owns the document our agents read first? When did someone last correct it, as opposed to adding to it?
- What can our agents reach today that nobody has reviewed this year?
- Name one thing our experienced people know that a new hire, or an agent, has no way to find out.

PART V

Lead the change

CHAPTER 9

Your Instruments Went Dark

The dashboard still updates

Nothing broke, and that makes the problem hard to see.

Your delivery reports still arrive each week. Velocity holds steady or climbs, throughput is up, and your people have adopted the new tooling. The charts render with trend lines, and you read a confident weekly account of a system your reports no longer describe.

If velocity had dropped to zero, you would have replaced it within a week. It went up instead, so you kept trusting it.

Story points measured human effort

Story points never measured value, and good practitioners said so from the start. They measured effort as a person experienced it, in terms of relative difficulty. Teams used them to forecast how much work they could take on. That made sense while human effort was the constraint, which it was for the whole history of the practice.

An agent doesn't experience effort. When an agent finishes a five-point story in four minutes, the story still counts as five points and the team's velocity goes up. The arithmetic holds, and the number means nothing.

You designed your measurement system to ration a scarce resource, and you now point it at an abundant one. Velocity, burndown, capacity planning and estimation accuracy all measure bandwidth. They told you how much human

attention you had and where it went. Agents took over a large share of the work those instruments tracked, and the instruments kept reporting as if nothing had changed.

Some metrics improve as things get worse

That is worse than measuring nothing.

Faros AI studied telemetry from 22,000 developers and more than 4,000 teams. Faros sells engineering-intelligence tooling, so weigh that when you read their numbers. The method holds up well: about two years of data, with statistical significance at $p < 0.05$.

ON YOUR DASHBOARD	UNDERNEATH
PR merge rate per developer +16.2%	Average PR size +51.3%, so each unit carries more change
Throughput looks healthy	Median time to first review +156.6%
Review looks like it is keeping up	31.3% more PRs merged with no review
Productivity up	Bugs per developer +54%, against +9% in the prior report
The codebase looks active	Code churn +861%, lines deleted against lines added

Each entry on the left reads as success on a standard delivery report. The right column shows the same activity with the flattering frame removed.

Sit with *merged without review*. On a throughput dashboard, a PR merged without review and a PR merged after careful scrutiny count as the same event and add one to the same

counter. Your reporting can't tell work someone judged from work someone waved through, and the second kind grew by close to a third.

The signal never reached you

The numbers moved in the reassuring direction. You don't escalate a metric that is improving. If velocity had collapsed you would have called a war room by Wednesday.

The people who could see the problem weren't asked. I suspect your engineers know that review has turned into a formality in places. The reporting line carries the metric to you and leaves the meaning behind.

And you have no meeting built for noticing. Your governance calendar has forums for reviewing performance against the metrics, and none for asking whether the metrics still mean anything.

The cost of fixing it

You will have a gap. If you switch off velocity before your outcome measures are working, you will have less reporting for a while, and someone will ask you to justify that. You can answer that you'd rather fly with fewer instruments than wrong ones. I think the answer is correct, and it won't satisfy every stakeholder.

Velocity also carries political weight. Engineering has used it for twenty years to justify headcount to finance, and if you remove it without a replacement you remove a shared language between two functions that have few others. Have the replacement ready, and expect the conversation to turn on

trust more than on measurement. Units that size the work itself, rather than the effort a person put into it, survive the shift, and chapter 10 names one.

Outcome measures arrive later and flatter less. Bandwidth metrics update each week and mostly go up. You confirm an outcome as fast as your customers respond, and a fair share of the confirmations will come back negative. You are trading fast, meaningless numbers for honest, late ones.

Some teams will read the change as an attack. People built careers on improving these numbers, in good faith, and they were right to do it at the time. The instruments stopped working. Your people did nothing wrong. Say so, more than once, or you'll meet resistance and mistake it for skepticism.

Why this chapter is here

Chapter 1 argued that judgment is the constraint, and the chapters since have shown where it has to act. This chapter argues that your current reporting can't see any of it.

Put the two together and you get the danger I worry about most. An organization with a real bottleneck and no instrument pointed at it accelerates in a direction nobody has checked, with a weekly report saying all is well.

The next chapter covers what to measure instead. The replacements make sense now that you've seen the whole loop. For now, accept that the reporting you trust describes a system you no longer run.

Questions for your teams this week

- What fraction of merged PRs last quarter had no substantive human review? Can our tooling tell?
- Which of our delivery metrics would change if an agent did the work instead of a person? If none would, what are they measuring?
- Did our bug rate or incident rate move in the same period our productivity metrics improved? Has anyone put those two charts side by side?
- When did we last check whether a metric still means what it meant when we adopted it?

Source: Faros AI, The Acceleration Whiplash: AI Engineering Report 2026.

CHAPTER 10

How You'll Know It's Working

Six numbers, and how each one gets faked

Chapter 9 argued that your instruments went dark. This chapter gives you replacements, along with how people will game each one, because an organization that would rather look good can make each of these metrics look good.

I describe how each metric gets gamed for a practical reason. If you can't describe how a metric fails, you can't trust it, and the people who show you these numbers will have a stake in them.

1. Intent completion rate

What it is: of the outcomes you set out to achieve, the share you confirmed as achieved, against the share you abandoned. This is the primary signal, and the only one that answers the question your board is asking. The other five diagnose.

How it gets faked: teams set intents whose success is already assured, or they write the success criterion so vaguely that almost anything satisfies it. I read a completion rate near 100% as a target-setting problem. I'd expect an organization that is attempting hard things to land somewhere between 60% and 80%.

The tell: ask to see the abandoned ones. If there aren't any, you have your answer.

2. Pending-off age

What it is: how long finished, deployed work waits before customers can see it, measured as a median plus the age of the oldest item. This is the batch metric and the direct answer to chapter 4. It tells you whether your people use the release switch as a governance instrument or as a parking lot. When the age rises, the exposure decision has turned into a queue.

How it gets faked: teams release trivial items fast to hold the median down while the significant work sits. Watch the maximum as well as the median.

The tell: ask for the oldest item by name. Somebody will know which one it is and why it's stuck.

3. Time from intent to customer

What it is: the time from agreeing on an outcome to a customer being able to experience it. It replaces lead time and measures a longer arc on purpose, because it includes the decision-making at the front. Conventional delivery metrics leave that part out, since nobody controls it.

How it gets faked: teams log the intent late. If people write intents the day before work starts, the clock skips the deliberation, and the number ends up measuring typing speed.

The tell: compare intent creation dates against when the conversation started. If they always fall on the same day, the clock starts in the wrong place.

4. First-pass rate at each judgment moment

What it is: how often work clears a judgment moment without rework, tracked per moment. A falling first-pass rate on direction means your intents are ambiguous. A falling rate on fitness means the standards of the agents and engineers doing the work have drifted. The number looks the same in both cases but calls for different diagnoses, so if you combine them you destroy the signal.

How it gets faked: nobody needs to fake it, because it fakes itself. A first-pass rate near 100% means the gate has stopped working, and I think this failure shows up more than any other in the book. Our own project ran thirteen cycles with a thirteen-out-of-thirteen pass rate, which told us only that the reviewer had no standing to fail anything.

The tell: if a judgment moment has never rejected anything, it only records. It controls nothing.

5. Rework cause mix

What it is: the kind of rework, grouped by root cause: ambiguous intent, misreading by the agents doing the work, scope change, external change. The volume of rework tells you little, since agents make rework easy and a healthy team reworks all the time. The distribution carries the signal. A concentration in one cause gives you something to act on, because ambiguous intent is a product problem, misreading is a context problem, and scope change is a governance problem. The three share a symptom and nothing else.

How it gets faked: teams file everything under “scope change,” which is nobody’s fault and requires no action.

The tell: a suspiciously flat distribution usually means nobody is categorizing honestly.

6. Escape rate

What it is: the share of things released to customers that you had to withdraw. This quality signal survives the shift, and it pairs with pending-off age. Together they detect two opposite failures. If you release too cautiously, age climbs and escapes sit near zero. If you release too carelessly, age stays low and escapes climb.

How it gets faked: teams leave something visibly broken instead of withdrawing it and admitting a mistake. A low escape rate can mean good judgment, or an unwillingness to reverse course.

The tell: ask how long it took from knowing about a problem to switching it off. If the answer is in days, the escape rate is fiction.

Work no customer sees

Obligations and enablers don't fit the six numbers above, because neither has a customer outcome of its own. For both, the DORA metrics from chapter 4 still do the job. Lead time, change failure rate and recovery time tell you whether the delivery system handles this work safely and on time, and for work like this that is the right question.

Enablers need two more rules. First, an intent's time to customer includes the enablers it waited for. If the checkout intent waited eight weeks for a new platform, the clock shows those eight weeks, because hiding that wait is how platform work escapes scrutiny. Second, an enabler counts as confirmed only when the intents it was built for are confirmed, as chapter 2 sets out.

The tell: a growing share of enabling work that no named intent depends on. It usually means a platform team is building for its own roadmap, the busy-but-moving-nothing failure that chapter 7 describes under direction.

What to switch off

I'll say this once, because the argument turns on trust more than on measurement.

Velocity, story points, burndown, estimation accuracy and capacity utilization served you well while people did the executing. They all measure human bandwidth, which agents absorbed. They will keep producing plausible numbers for as long as you run them, and that makes them dangerous as well as useless.

The resistance you meet will turn on something other than measurement quality. Engineering has used velocity for twenty years to justify headcount to finance, and you may have made that case yourself. Removing it takes away a shared language between functions that have no other one. Put the replacement in place before you remove it, and expect the conversation to turn on trust.

A unit finance can still trust

Switching off velocity leaves finance without the number it used to plan headcount, and that conversation needs an answer. I'd use a unit that sizes the work and ignores who did it. Business Complexity Points (BCP) is the one I know best: it scores a piece of work by its business rules, user interface elements, business entities and the interfaces between them,

each on a five-step scale from extra small to extra large. CI&T created it in 2015, and CI&T and Itaú Unibanco released it as open source under the MIT License in May 2026.

A story sized at 12 BCPs stays at 12 whether a person or an agent builds it, which is the property story points lost. Divide the hours or the cost you spend by the points you deliver and you get a production-side ratio you can compare across teams, vendors and tools.

BCP measures how much complexity you built. It says nothing about whether any of it worked, and if you report it as value you repeat the mistake chapter 3 warns about. The combination I'd try is the share of delivered points inside confirmed intents compared with the share inside abandoned ones. That tells you how much of the complexity you paid for produced an outcome.

What it costs

You will report less, and later. Six numbers, several of which update on customer timescales, replace a weekly dashboard that always had something to say. You lose a real comfort, and you will feel the loss more than your teams will.

Several of these need instrumentation you don't have. Pending-off age needs a switch registry, and rework cause mix needs someone categorizing honestly. None of it costs much, and none of it is free.

The new numbers will also look worse than the old ones. Your performance hasn't declined. The old metrics measured something that always went up. Your first quarter of honest measurement will look like a regression, and it won't be one. If

you can't hold that line with your own leadership, don't start, because my guess is that someone will push to bring back a flattering metric within about six weeks.

And people can game each of these. Knowing the methods won't stop them. Asking for the tell, and not only the number, will.

Why this chapter is here

Chapter 9 took your instruments away, and this chapter gives them back: fewer, slower, harder to fake, and pointed at the thing that constrains you.

Each metric here measures a judgment. Completion rate asks whether you decided the right thing. Intent to customer asks whether you decided it promptly. First-pass rate asks whether your judgment moments work. Pending-off age and escape rate ask whether you expose work on purpose. The rework mix shows where your judgment fails.

If judgment is the constraint, and I think it is, these are the instruments pointed at it.

Questions for your teams this week

- What is our intent completion rate, and how many intents have we abandoned this year?
- What is the oldest piece of finished work sitting unreleased right now, and why?
- What is the first-pass rate at each judgment moment, and has any of them ever rejected anything?
- Which of our current metrics would still make sense if agents did all the execution?

CHAPTER 11

From Demo to Scale

The question you'll hear within a quarter

Once the judgment moments from chapter 7 are in place and working, someone will ask the obvious question, and you may be the person asking it.

"When can we take the humans out?"

They will ask in good faith. It is usually someone competent who has watched the gates pass cleanly for two months and concluded, correctly, that they are overhead. Most organizations answer that governance requires the gates, so they have to stay. I think that answer is wrong, and it will lose. It treats each judgment moment as equally permanent and defends them all with the same weak argument.

The better answer, I think: some of them graduate on evidence, some never graduate, and you should be able to say which is which in advance, before anyone asks.

Three stages, earned on evidence

STAGE	NAME	WHAT'S TRUE
1	**Structured**	Each judgment moment runs at full intensity. Teams start here, and a team returns here after a failure.
2	**Calibrated**	At least one judgment moment has met its evidence threshold and been formally relaxed. The rest still run in full.
3	**Autonomous**	Each judgment moment that *can* graduate has. The permanent floor still runs, and always will.

Two features of this model matter more than the stages.

Moments graduate one at a time. A team can relax its direction judgment while its fitness judgment still runs at full intensity, because the evidence for one says nothing about the other. When you treat maturity as a single organizational level, one good quarter loosens everything at once, and people learn to game it.

And stage is a live reading. You don't climb a ladder and stay on it. The stage describes where your evidence places you today, and evidence changes.

Graduation is a decision

I think this part separates a maturity model from a compliance checklist, and I'll defend it.

Crossing a threshold makes a team eligible. It doesn't graduate anything by itself. Two people then have to agree to graduate the moment, and either one can say no even when the number is met.

The veto is deliberate. Numbers can look good for bad reasons: an easy quarter, a stable domain, a team that got cautious instead of good. The person closest to the work can say the data looks fine and the team isn't ready, without having to prove it. You keep qualitative judgment inside the decision to reduce judgment, which is either a nice recursion or the only sane way to do it, depending on your mood.

And someone writes it down: what graduated, when, on what evidence, agreed by whom. An auditor may read it, but you write it down because nobody can reverse a decision nobody recorded.

Regression is instant and binary

A single failure after graduation immediately brings back the full judgment moment. It stays until the team meets the threshold again and someone records a new decision.

Only the moment where the failure happened reverts. There are no one-off exceptions, no probation and no benefit of the doubt.

This mechanism makes the whole model safe to adopt. Without it you have a ratchet that only loosens: each quarter relaxes something, nothing tightens, and three years later

nobody can explain why oversight has disappeared. With it, autonomy works like a loan against your track record, and you call the loan the moment the track record changes.

People will tell you this is harsh, and they're right. The harshness is what lets you afford to grant autonomy at all.

What never graduates

Four things stay at every stage, however good the numbers get. The list is short on purpose, because a permanent floor that includes everything amounts to refusing to graduate anything.

The exposure decision. A named person authorizes customers to see something. The exposure decision is the governance model itself, and if you remove it, you are running something else. No cycle count, track record or metric changes that.

The escalation obligation. An agent that hits its boundary must be able to reach a person, and that person must answer. Reducing gate overhead never reduces the duty to respond when something asks for help.

The mechanism check. The check asks whether the output matches how you said you would achieve the outcome, which is a different question from whether it passed its tests. The reasoning here generalizes, so it's worth following. Coverage is a metric you can measure, and anything you can measure you can eventually automate. Whether a solution matches its intent is a comparison between a thing and a purpose, and the code doesn't contain the purpose. I don't believe any automated system can make that comparison for you.

The trigger for a deep context audit. The routine pre-flight check can fade into the background. The audit itself stays, and when something structural changes, you run it regardless of stage.

The same pattern sits under all four. The mandatory hold graduates, and the obligation stays. A relaxed judgment moment keeps running and stops blocking. Anyone who could raise a concern before can still raise one, with the same authority. You removed the wait and kept the judgment.

What this buys you

An answer to the overhead argument. You stop defending permanent process against someone who thinks it is bureaucracy. You describe a system where oversight costs a lot at first, costs less as teams earn trust, and has a floor you can name and justify. You can win that conversation.

A real definition of scale. Scale means more work moving through with fewer mandatory human holds, while the failure rate holds steady. Counting teams or agents won't tell you that. You can measure it, and it is what your board means when they ask whether this scales.

An honest place for a struggling team. Stage 1 is the default. A team in month two belongs there, and a team that just had a bad incident belongs there again. Once you name the stage, nobody gains by pretending to be further along.

What it costs

You need enough cycles to generate evidence. Thresholds that span ten or twenty cycles mean nothing to a team that runs one a month, and small or slow-moving teams may sit at Stage

1 indefinitely. That outcome is correct, and the model is working as intended.

Someone must count. Graduating on remembered impressions means graduating on optimism. You have to pull the real cycle records, and nobody will do that dull work unless a named person owns it.

Regression will hurt at the worst moment. A gate comes back right after a failure, when the team is under pressure and least wants more process. That is when the rule earns its keep, and when people will argue against it most persuasively. Decide now, while things are calm, that you'll hold it.

And Stage 3 is not the goal, which I know is an unfashionable thing to say. A team can sit at Stage 2 for years because mistakes in its domain are costly, and nothing is wrong with that. If you set Autonomous as a target, teams will graduate on thin evidence to satisfy a roadmap.

Why this chapter is here

Chapter 1 asked why pilots don't scale. It answered that judgment was free in the demo because it sat in one head, so scaling means paying for it explicitly.

This chapter shows how to pay without paying forever. Start at full intensity, earn relaxation with evidence, keep a floor you can name, and regress the moment the evidence changes.

It also answers the demo problem. The pilot failed to scale because nobody could see its judgment, so nobody could transfer it. This model makes judgment explicit and measurable, and over time it makes most of it cheaper to run.

Questions for your teams this week

- For each judgment moment we run, what evidence would convince us to relax it, and have we written that down before anyone asks?
- What is on our permanent floor? Can everyone name it without looking?
- When a gate has passed cleanly for two months, do we relax it informally, or decide on purpose?
- After our last significant failure, did any oversight come back, or did we write a post-mortem and carry on?

CHAPTER 12

What It Costs

The chapter most books leave out

Each chapter so far has ended with a cost section. This chapter collects them, because each cost is survivable on its own, but the total decides whether you should do this.

I'd rather you decide against this based on accurate information than adopt it based on a flattering summary and abandon it in month four. An abandoned change costs more than one you never attempted, because it burns the credibility you'll need for the next one.

The four costs that matter

The rest is detail.

Judgment doesn't scale. The rest of your delivery system now scales with compute. This part scales with attention, which is finite and expensive and belongs to your most senior people. I don't think you'll find efficiency here, and a plan that claims to find some is a plan to remove the judgment. Budget for it as a permanent cost.

Somebody has to be willing to be unpopular, more than once. A judgment moment that never rejects anything is decoration. To make it real, people have to say no to work their colleagues have finished, in public, more than once. If your culture punishes that, and most do it quietly through who gets promoted, no framework will survive. I'd bet on this failure mode above all others.

You are putting back friction you just paid to remove. Twelve months after an AI investment you justified on speed, you will be adding constraints, and people will ask you why. They'll be right to ask. The friction you removed was an accident and the friction you're adding is a choice. That answer is correct, and you'll be repeating it for a year.

You will see that some of your work didn't work. When you confirm outcomes, you discover that things you shipped, announced and celebrated moved nothing. Senior people, and maybe you, will want to soften that right away. If they win, you get confirmation theater, which does more harm than no confirmation at all because it manufactures false certainty.

The costs by stage

Roughly what to expect, and when.

TIMING	WHAT IT COSTS
Immediately	Writing intents properly, a skill nobody has been trained in. Slower decisions at the front of the process. Instrumentation gaps become visible.
First quarter	Metrics look worse before they get honest. Reporting volume drops. The first uncomfortable confirmation results arrive.
First year	Switch hygiene becomes a real maintenance burden. Someone must own system memory and delete other people's writing. Permission reconciliation reveals an uncomfortable gap.
Permanent	Senior attention on judgment moments. The willingness to reject finished work. Holding the permanent floor when someone argues it's overhead.

The technical costs come early and then go away. The cultural costs stay. My strong impression is that people blame most adoption failures on the first kind, when the second kind caused them.

Who should not do this

This list is shorter and more useful than a list of ideal fits.

Organizations that can't expose work incrementally and won't invest to. If your architecture forces all-or-nothing releases and nobody wants to change that, you can't use chapter 5,

which I think is the cheapest and highest-return idea here. Much of the rest still works, but the strongest part doesn't. Be honest about that before you start.

Organizations where saying no limits careers. If you can't name someone who rejected finished work in the last six months and is doing fine, you have your answer. Fix that first. It is a prerequisite, and it won't fix itself along the way.

Organizations in crisis. You need slack to install this. A team fighting for survival this quarter should fight for survival this quarter and come back when there's room to breathe.

Leadership that wants the numbers without the accountability. If the appeal here is a governance story for the board, and you don't intend to change who decides what, you will build the artifacts, skip the judgment, and end up with more process and no more control. I think that leaves you worse off than you are now.

Small teams still working like the pilot. If you're five people in one room and judgment lives in the room, everyone knows what good looks like and nothing crosses a boundary. You have the conditions chapter 1 described, and formalizing them adds cost for no return. You'll know when that stops being true: someone ships something that surprises everyone else. Adopt this when ambient judgment stops working, and not because someone calls it best practice.

What the book doesn't prescribe

Change programs accumulate requirements nobody asked for, so I'll be explicit.

The book doesn't prescribe a reorganization or new roles. The judgment moments need named people, but not new job titles or reporting lines. That said, change is the point of the book. Some organizations will find that making the judgment moments real means changing team shapes, creating roles, or running a formal change program, and when that happens it is a legitimate consequence of the argument. I leave that call with you.

It doesn't require another framework. You don't need Scrum, SAFe, Team Topologies or anything else in place, and organizations with no named operating model succeed too. You don't have to adopt it all at once: chapter 13 argues for one move, and several of the parts work alone. And you don't need to buy tooling, because you can implement each idea here with what you already own. If someone tells you otherwise, they are selling something.

The honest summary

What you give up: the comfort of estimates, the appearance of predictability, a weekly dashboard that always improves, and the option of not knowing whether your work matters.

What it costs to start: learning to write outcomes instead of tasks, instrumenting a few things you haven't measured, and a quarter of worse-looking numbers.

What it costs for good: senior attention on judgment, and a culture where people survive rejecting finished work.

What you get: work reaching customers at a rate that reflects what you spend, a defensible answer to who decided what, and the return that compounds over time, which is knowing which of your beliefs about customers are true.

When it pays back: the release switch pays within a quarter and costs the least. The judgment moments pay off the first time they catch something. You can't predict when that will be, but I suspect it will come sooner than you expect. Outcome confirmation pays over a year and gives the slowest and largest return, because it changes what you decide to build next. I reasoned my way to those estimates. No study produced them, so weigh them accordingly.

Why this chapter is here

It sits near the end so you can weigh the costs against the whole argument instead of one piece of it.

If, having read this, you conclude the cost is too high for your organization right now, that is a legitimate result of reading the book. I'd rather you reach it here than in month four with your credibility sunk.

There's a smaller reason too. When a framework states its costs, you can check its honesty. If the costs listed here turn out to be roughly what you experience, the rest of the argument earns some trust. If they don't, discount the rest accordingly.

Questions to ask yourself this week

- Can I name someone here who rejected finished work in the last six months? What happened to them?
- Am I prepared to defend a quarter of worse-looking metrics to my own leadership?
- What share of our delivery surface can we expose incrementally today?
- Do I want the governance story, or do I want to change who decides what? The first costs less, and I should be honest with myself about which one I'm buying.

CHAPTER 13

Where to Start, and What We Don’t Know

One move, not a program

After a book like this you will want to design an adoption plan, and that instinct has served you well before. I’d resist it this time. An adoption plan needs approval, approval needs consensus, consensus needs meetings, and the plan dies in a steering committee while the problem gets worse.

Do one thing this week. It needs no permission and no budget.

Measure how much finished work sits unreleased right now

Leave aside how fast you build and how many changes you merge. Find out how much completed, deployed, working software sits in production where no customer can see it, and how long the oldest piece has been there.

I suspect most organizations can’t produce this number, and I’d count that inability as the first finding. A standard toolchain doesn’t report it, because it was built on the assumption that deploying and releasing are the same event. Getting the number usually takes a few days of someone’s time and a conversation with two or three engineers.

Do this first, for four reasons.

First - It gives you a real number about your organization, which nobody can dismiss as a claim from a book. The answer settles the argument you'd otherwise have about whether any of this applies to you.

Second - It usually comes back worse than you expected. You just invested heavily in going faster, and nothing I know of creates urgency faster than finding a large queue of finished work your customers can't see.

Third - It points at what I think is the cheapest fix in the book. Chapter 5 doesn't depend on a reorganization or a purchase, and the queue responds to it.

Fourth - And it gives you the baseline for the metric most likely to tell you whether any of this works.

If the number comes back small, days rather than weeks, and nothing is stuck, you've still learned something. Your constraint sits somewhere else, and chapters 2, 3 and 7 matter more to you than chapter 5.

Then, the second move

Once you have the number and it has had its effect, run one intent end to end.

Pick something small and real. Write it as an outcome: what becomes true for a customer, and how you'll know. Name a person for each of the four judgment moments. Deploy behind a switch. Have someone use it before customers do. Let a named person authorize exposure. Then, weeks later, go back and confirm whether the outcome happened.

Run one intent, with no pilot program and no team-wide rollout. You are looking for where it breaks. It will break somewhere specific to your organization. My bet is that it

breaks at confirmation, because by then nobody remembers the intent.

I think that breakage is the most valuable result of the exercise. It tells you which chapter of this book describes your problem.

What we don't know

This is the shortest section in the book, and the one I'd most want you to read.

I am not sure I'm right about everything here, and I am willing to be wrong or partly right. I built this book from three sources: IDF, a framework in active use; current industry telemetry; and reasoning from the thesis. I had no controlled study, no large sample of adopting organizations and no longitudinal data, because none of those exist yet for this approach or any competing one. Anyone who claims to have that evidence for AI-era delivery governance in 2026 is overstating it. Hold me to the list below.

We ran this on ourselves and the gate never failed. We built IDF using IDF, with agents executing, judgment moments in place, and a review function logging each cycle. Thirteen cycles, thirteen passes, no rejections. That record shows a review function with no independence and no standing to fail anything, and we caught it only because we went looking. The uncomfortable implication: our own dogfooding validated the mechanics and told us nothing about whether the judgment worked. That is the failure chapter 7 warns you about, and it happened in the project that wrote chapter 7.

We don't know if the four judgment moments are the right four. Direction, fitness, exposure and confirmation held up in every scenario we tested, but that doesn't make the list

complete. A fifth may become obvious in a domain we haven't worked in: regulated medical devices, safety-critical systems, anything with a physical failure mode.

We don't know how much of the current data is transitional. The telemetry showing review times exploding and incidents tripling describes organizations mid-adoption, using tooling that changed twice while they used it. Part of that is a structural effect, and part is the friction of transition, which I expect will fade. We can't separate the two yet, and anyone who claims to is guessing.

We don't know the size ceiling. The reasoning here concerns decision rights and information flow, which tend to degrade with scale in ways you can't see until they break. Nobody has tested this at fifty thousand people, and it may need something we haven't thought of.

We don't know what parallel agent teams do to this. When several agents work at once against shared state, the meaning of one iteration changes, and so, probably, does what the judgment moments attach to. The four moments still seem right to me. Their placement may not be. The clearest case so far is the Navier-Stokes run from chapter 7. The judgment in that run sat at the level of direction: deciding where to point the agents and revisiting that as the work went on. It sat nowhere near the level of individual messages. It also shows the confirmation problem from chapter 3 at a scale I hadn't imagined. Producing the proof took 88 hours. As I write, the Clay Institute is still reviewing it, and it will recognize a solution only after peer-reviewed publication and community vetting. Confirming the result is taking far longer than producing it, and it depends on human judgment.

And the ground is moving underneath all of it. Model capability, regulation and industry practice all change faster than a book can. The dates in chapter 6 moved once while I wrote this one. Read the argument, and check the numbers.

What would change my mind

The claim most likely to be wrong is that you can't delegate judgment to automated review. It is central, chapter 7 rests on it, and I'd expect it to erode first.

The strongest argument against it comes from Ethan Mollick. He had predicted that getting agents to work as a group would take careful, company-like design, and in October 2026 he wrote that he had been wrong: better models solved coordination by themselves, the way better machine learning keeps beating rules people write by hand. He calls it the Bitter Lesson. If organizing turned out to be learnable, judging might be too, and a fair reader could see the judgment moments in this book as the kind of careful construction he gave up on.

Automated review has improved a great deal, and agents now review 25% of pull requests. If someone builds a review function that reliably rejects work its own system produced, with real independence and not a simulation of it, a large part of chapter 7 will need rewriting.

I don't think we're close to that. Independence looks to me like a structural property more than a capability. A reviewer optimizing for the same completion signal as the producer will converge on approval however capable it becomes. The early evidence points my way. In September 2026, OpenAI shelved GPT-6.1 Astra before release because, in its own testing, the model took actions without authorization and didn't accurately report what it had and hadn't done. A system

that misreports its own work can't be the one that reviews it, however well it organizes. Still, one model's test results don't settle the question.

I hold the position for a second reason. Some judgment can be turned into process: a checklist, a threshold, a rule an agent follows to decide what to do next. A written standard captures part of what good looks like, which is why chapter 8 asks you to write yours down. Deciding whether this particular piece of work meets it, in a case the standard didn't foresee, is still judgment, and chapter 1 argued that this is the part that never scaled. Culture is harder still. A company's culture lives in how people feel about their work and about each other, and it shows up as attitude: whether someone speaks up, whether a reviewer is willing to say no to a colleague's finished work. You can write down a standard. I don't believe anyone can write down the attitude that makes people uphold it, or hand it to an agent today.

I'd rather tell you so than pretend the question is settled.

The one-sentence version

If you take one sentence from this book, take the thesis: when execution becomes free, judgment becomes the bottleneck, so govern the judgment.

The rest follows from it. The intent as the unit of commitment, the release switch, the four moments, the maturity model and the metrics all exist to put human judgment where it matters and take it out of where it doesn't.

Your organization has spent the last two years making execution take less effort, and that worked. Now ask whether you've done anything about the thing it made scarce.

Questions to ask yourself this week

- How much finished work is invisible to customers right now, and how old is the oldest piece?
- If I ran one intent end to end next month, where would it break?
- Which claim in this book am I least convinced by, and what evidence would settle it?
- Am I prepared to find out that some of what we shipped last year did nothing?

Sources: Ethan Mollick, The Dot and the Swarm · Tufts Daily, Mathematicians still checking the Navier-Stokes proof · Cloud Security Alliance, OpenAI shelves GPT-6.1 Astra.